

Házi Feladat

Programozás alapjai II.

Feladatspecifikáció

Balogh Tamás
QHIG5K

2026. március 31.

Tartalom

1. Feladat	3
2. Feladatspecifikáció	3
3. Játékszabályok.....	3
3.1 A tábla alaphelyzete.....	3
3.2 A bábuk lépésének szabályai	4
3.3 A játék vége.....	4
4. Terv.....	5
4.1 Osztálydiagram.....	5
4.2 Fontosabb függvények/algorithmusok.....	6
4.2.1 Piece::isValidMove	6
4.2.2 GameMaster::processMove	6
4.2.3 Renderer::display	7
4.2.4 Board::isPathClear	7
4.2.5 Board::isChecked	8
4.2.6 Board::isCheckedMate	8
4.2.7 PGNHandler::parseFile	8
4.2.8 PGNHandler::generatePGN	9

1. Feladat

A feladat lényege C++ nyelven objektum orientált paradigmákkal megvalósítani egy olyan parancssoros interfészen keresztül játszható sakkjátékot, amely ezen felül képes általános PGN formátumban megadott akár harmadik félnél (chess.com, lichess.org, etc...) játszott régebbi játékokat, vagy ezen program által generált játékokat visszajátszani. Minden véget ért meccs végén – legyen az lejátszott, vagy akár félbeszakított – megadja az addigi lépésekből álló PGN-t.

2. Feladatspecifikáció

A feladat egy olyan sakkjáték elkészítése, amely képes a játék szabályait betartatni. Ennek alapjául két atomi osztály fog társulni, egy a táblát kezelő osztály mely kezeli a tábla állapotát, és végrehajtja a lépéseket, ha azok lehetségesek. A második pedig egy minden bábuhoz tartozó őosztály, szimplán csak bábu, amely tartalmazni fog egy tisztán virtuális függvényt, amely a lépés logikáját valósítja meg.

A [fentebb](#) említett oldalakon van módunk változtatni a visszajátszott játékok menetén, ez most itt nem lesz lehetséges, viszont a lépések között oda-vissza lehet majd lépkedni, és azt személyesen elemezni.

A program rendelkezni fog egyetlen egy opcionális parancssoros bemenettel, az pedig vagy egy a PGN-t tartalmazó fájl elérési útja, vagy maga a PGN lesz. Ezek közül kapcsolókkal (*flag*) lehet majd választani.

3. Játékszabályok

3.1 A tábla alaphelyzete

A játékszabályok megegyeznek a sakk játékszabályaival. Két játékos játszik egymás ellen, az egyik a fehér míg a másik a fekete színnel. A játék első lépését a fehér színnel rendelkező játékos kezdi. Mindkét játékos rendelkezik 16 bábuval, melyek a közvetlenül előttük lévő 8-8 sorban helyezkednek el. Az előttük lévő első sor áll a tábla két szélén álló bástyákból, a mellettük álló lovakból, az azok mellett álló futókból, majd következik a király és a királynő. A király mindig a saját színén kezdi a játékot. Az előttük lévő 8 sor tartalmaz 8 gyalogot.

3.2 A bábuk lépésének szabályai

Gyalog: A gyalog (paraszt) ha még nem lépett léphet kettőt, egyébként csak egyet. A gyalog átlósan 1 cella távolságban tud ellenfélhez tartozó bábut elfogni. A gyaloghoz tartozik egy speciális lépés, az „*en passant*” mely akkor lehetséges, ha az ellenfél másik gyalogja úgy tudná kikerülni a gyalog leütését, hogy az első lépése által nyújtott kettős lépést felhasználja. Ilyenkor a kérdéses gyalog tud átlósan lépni arra a helyre, amit az ellenfél gyalogja éppen kikerült, ezzel őt elfogva.

Futó: Minden játékos rendelkezik két ellenkező színű futóval, ezek csak a saját színükön tartózkodhatnak a játék végéig, mivel csak átlósan mozoghatnak, de az átlókon egy irányban egy lépésben bármennyit.

Ló: A ló abban speciális, hogy átugorhat más bábukat, és L alakban léphet. Kettőt a saját során/oszlopán, egyet pedig jobbra vagy balra.

Bástya: A bástya bármennyit léphet előre, viszont a sorában vagy oszlopában teheti ezt meg. A kettőt egyszerre nem.

Királynő: A királynő a bástya és futó kombinációja. Egyszerre tud lépni bármennyit a sorában, oszlopában, és átlósan is. A megkötések ő rá is tartoznak.

Király: A király bármely irányban tud egyet lépni, ha azzal a lépéssel nem kerülne „*sakkba*”. Akkor van a király sakkban, ha egy ellenfél bábuja el tudná őt fogni. Ilyenkor ez egy illegális lépés melyet nem lehet megtenni. Ha a király sakkban van, akkor muszáj ellépnie az elől. Ezen felül a királlyal és az egyik bástyával lehet „*sáncolni*”, amely annyit takar, hogy ha még sem a bástya, sem a király nem lépett, és a kettejük között lévő összes bábu már ellépett, akkor helyet tudnak cserélni a királyt kimenekítve a tábla közepéről. Fontos, hogy sakkon keresztül nem lehet sáncolni.

3.3 A játék vége

A játéknak akkor lehet vége, ha az egyik játékos királya már nem tud ellépni a sakk elől, vagy nem tudja a játékos a sakkot egy másik bábuval megtörni. Ebben az esetben a sakkot adó játékos nyert. Ez a **matt**.

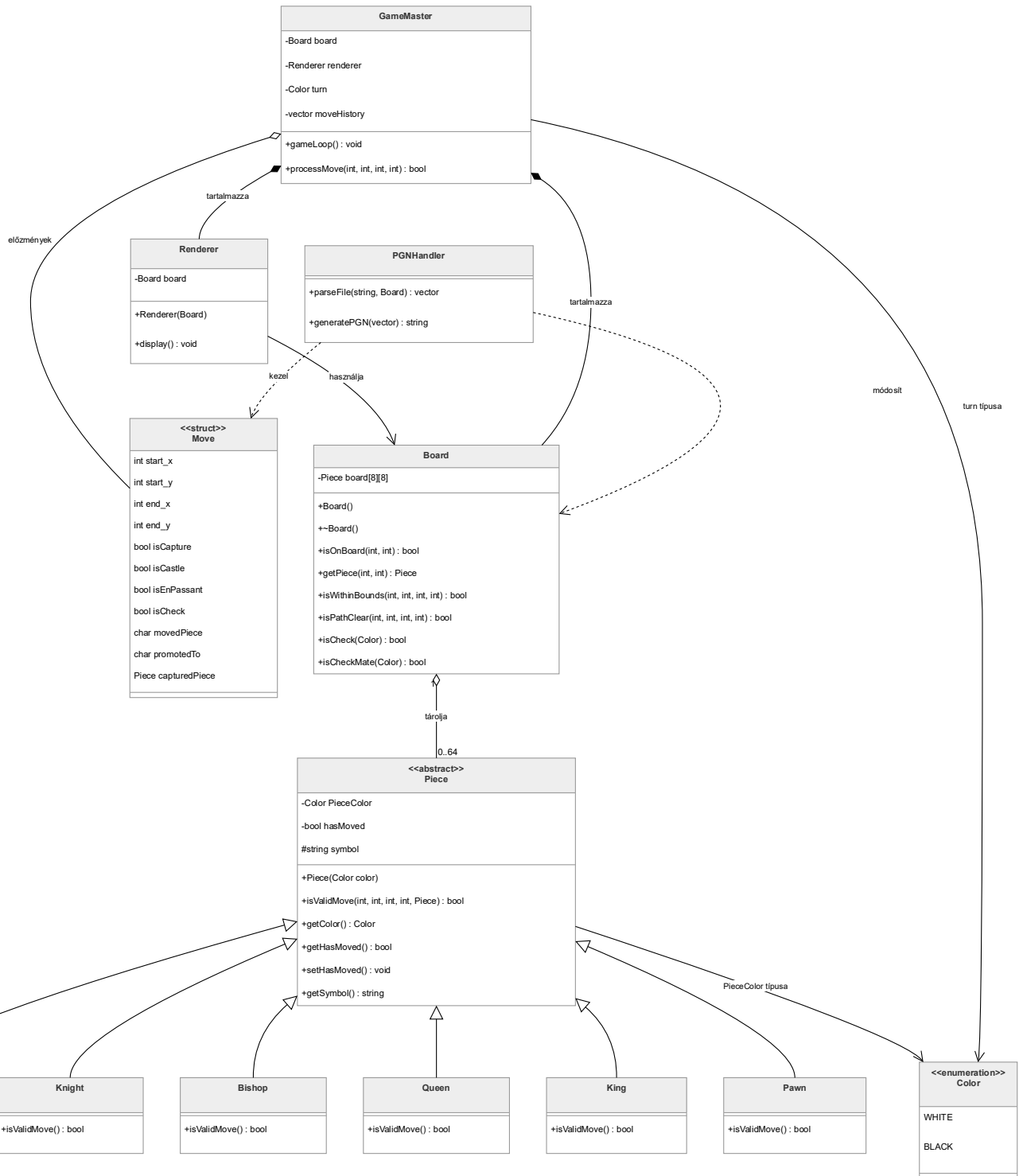
A játék lehet döntetlen, ha az egyik játékos nincs sakkban, viszont már nincs érvényes lépése, amivel nem tenné magát sakkba. Ilyenkor a játék döntetlen. Ez a **patt**.

Speciális játék vége lehet, ha 50 lépés óta nem történt sem gyalog lépés, sem bábu elfoglalás. Ilyenkor a játék döntetlenben véget ér. Ez az **50 lépéses szabály**.

Ennek testvére a **3 lépéses ismétlés**, mely kimondja, hogyha három egymás utáni lépésben (3-3) a két ellenfél ugyanazokkal a bábukkal oda vissza lépked, akkor a játék döntetlenben véget ér.

4. Terv

4.1 Osztálydiagram



4.2 Fontosabb függvények/algorithmusok

4.2.1 Piece::isValidMove

Ez a függvény egy tisztán virtuális metódus, amelyet minden bábu felül ír saját magának. Ez a bábuk szíve.

Általános algoritmus:

1. Megnézzük, hogy egyáltalán mozgott-e a bábu.
2. Kiszámoljuk az x és y tengelyeken történő elmozdulás abszolút értékét.
3. Minden bábu implementálja az ő rá specifikus geometriát az elmozdulás normál vektora segítségével.
4. Megnézzük, hogy a célmezőn álló bábu milyen karakterisztikával rendelkezik, tehát pontosabban: nem lehet leütni a saját színű bábút.
5. Ha a fentiek közül egyik kizáró feltétel sem teljesült, akkor igazzal tér vissza.

4.2.2 GameMaster::processMove

Ez a függvény a játék motorja, amely összeköti a játékost, a táblát, és a bábukat.

Algoritmus:

1. A bemeneti adatok alapján lekéri a kezdő koordinátán álló bábút.
2. Ellenőrzi, hogy van-e ott a táblán bábu, és az hozzá tartozik-e.
3. Lekéri a cél koordinátát, áll-e ott bábu.
4. Meghívja a vizsgált bábu isValidMove metódusát.
5. Ha igazat ad: Huszár kivételével minden bábuval meghívja a tábla isPathClear függvényét, hogy van-e akadály az úton.
6. Ha a lépés eddig minden ellenőrzésen átment, akkor szimulálja a mozgatót, megnézi, hogy a board.isCheck(saját szín) metódus mit ad vissza.
7. Ha hamisat ad vissza, akkor végrehajtja a mozgatót. (bábu áthelyezése, leütött bábu eltávolítása, lépés történetbe bevezetése).
8. Turnus váltása.

4.2.3 `Renderer::display`

Ez a függvény lesz az, amely ránéz a táblára és az általa szolgáltatott adatok alapján megjeleníti azt a képernyőre. Mivel a tábla logikailag egy 8x8-as kétdimenziós tömb, az algoritmus egy klasszikus mátrix bejárást valósít meg.

Algoritmus:

1. Kirajzolja a fejléctet (A-H)
2. Külső ciklus: Sorok bejárása, x koordináták 0-7 indexeken keresztül, minden sor elején kiírja a sor számát, csökkenő sorrendben.
3. Belső ciklus: Oszlopok bejárása, y koordináták 0-7 indexeken keresztül.
4. A ciklusmagban lekérdezi a `board.getPiece(x,y)` metódus meghívásával az adott cellában álló bábu memóriacímét.
5. Ha a visszkapott mutató nem `nullptr`, akkor ott áll egy bábu, melytől elkéri a `getSymbol()` metódus használatával a hozzá tartozó szimbólumot és azt rajzolja ki, ha pedig `nullptr` akkor az üres mezőt reprezentáló szimbólumot rajzolja ki.
6. A belső ciklus végén beszúr egy `std::endl` karaktert.
7. Lábléc kirajzolása, szimplán megismétli a fejléctet (A-H). Segíti az olvashatóságot.

4.2.4 `Board::isPathClear`

Ez az algoritmus ellenőrzi, hogy a csúszó bábuk útjában nincs-e más bábu.

Algoritmus:

1. Kiszámítja az elmozdulás irányát (`stepX`, `stepY`), értékük lehet 1, 0, -1 a tengelyek mentén.
2. Egy ciklussal elindul a kezdőpontból a végpont irányába, minden mezőt vizsgálva.
3. Minden mezőre meghívja a `board.getPiece(x, y)` metódust.
4. Ha bármikor nem `nullptr`-el tér vissza, akkor visszatér hamisként.

4.2.5 Board::isCheck

Az algoritmus paraméterként bekéri a jelenlegi soron lévő játékos színét, majd megnézi, hogy a király a sakkban van-e.

Algoritmus:

1. Megtalálja az adott színű király koordinátáit.
2. Végigmegy a tábla összes mezőjén.
3. Ha ellenkező színű bábut talál, megnézi, hogy az adott bábu isValidMove metódusa igazat ad-e vissza a király koordinátáira.
4. Ha az előző ellenőrzés igazat ad vissza, akkor megnézi, hogy szabad-e az út a király felé.
5. A bármelyik ellenséges bábu látja a királyt, és az út is tiszta, akkor a függvény igazat ad vissza.

4.2.6 Board::isCheckMate

Az algoritmus az isCheck algoritmust annyiban egészíti ki, hogy még azt is megnézi, hogy a király el tud-e lépni a sakk elől.

4.2.7 PGNHandler::parseFile

Ez a függvény felelős azért, hogy egy külső fájlból beolvassa a lépéssorozatot, és azt a program számára értelmezhető Move objektumok listájává konvertálja.

Algoritmus:

1. A megadott elérési úton lévő fájlt megnyitja és a tartalmát egy sztringbe tölti.
2. Megtisztítja a szöveget:
 - a. Metaadatok törlése
 - b. Megjegyzések törlése
 - c. Sorszámok törlése
3. A maradék tiszta szöveget szóközök mentén darabolja (tokenizálja)
4. Minden egyes lépésre elvégzi a következőket:
 - a. Bábu beazonosítása
 - b. Célmező beazonosítása
 - c. Forrásbábu keresése: végigiterál az adott típusú bábukon és meghívja rájuk az isValidMove metódust.
 - d. Szimuláció
 - e. Létrehoz egy Move objektumot és eltárolja egy vektorban
5. Frissíti a táblát.
6. Visszaadja a Move objektumokat tároló vektort

4.2.8 PGNHandler::generatePGN

A lépéstörténetet átalakítja szabványos PGN formátumú szöveggé.

Algoritmus:

1. Létrehoz egy üres kimeneti sztringet.
2. Hozzáadja a bábú betűjelét a movedPiece alapján (kivéve a gyalognál)
3. A belső indexet sakk jelöléssé alakítja
 - a. Ha isCapture igaz, akkor beszúr egy x-et
 - b. Ha isCastle igaz, akkor O-O vagy O-O-O-t szúr be
 - c. Ha promotedTo nem üres, akkor hozzáadja a célbábú jelét
 - d. Ha isCheck igaz, akkor beszúr egy + jelet.